

Living Showcase

Diese Ausgabe ist zugleich Präsentation, Bauteilkatalog und visueller Regressionstest. Jede Box zeigt sich selbst: Titel und Inhalt beschreiben, welcher Baustein hier steht und was er leistet.

Schnellzugriff

Welcher Baustein wo			Übersicht
Bereich	Kap.	Bausteine	
Aussagen	1	defbox (weight, tone), warnbox	
Verfahren	2	ZSFlist (ordered)	
Formeln	3	formulabox, goalbox	
Tabellen	4	ZSFtable, valuegrid	
Visuals	5	ZSFfig, figbox, splitbox	
Navigation	6	Marker, Verweise	
Code	7	codebox, CodeLine	
Stilmittel	8	Farbverfolgung, Klammern	

Lesart
Der rechts gesetzte Titel-Tag nennt jeweils die verwendete Umgebung, der Titel sagt, wofür sie da ist. So lassen sich Varianten direkt vergleichen und auswählen.

Kein Fachinhalt

Die Beispiele tragen bewusst keinen Lernstoff. Sie zeigen ausschliesslich das Verhalten der Bausteine — was hier steht, beschreibt den Baustein.

1. Aussagen und Hinweise

1.1. Titelboxen

runintext ist der ruhige Fliesstext-Baustein zwischen strukturierten Elementen. Er übernimmt automatisch die Lesbarkeitsregeln für schmale Spalten. Test des Umbruchschatzes: x: Erklärung für $f(x) = y$ mit 10 m/s und 95

Titelleiste und freier Inhalt

defbox

defbox trägt eine farbige Titelleiste und beliebigen Inhalt — gedacht für Definitionen und gewichtige Sätze. **So sieht die hervorgehobene Kernaussage einer Box aus.**

Dieselbe Box, leiser *weight=quiet*
weight=quiet tauscht die Kopfzeile gegen einen dezenten linken Balken — für kompakte Aussagen. Kein zweiter Baustein, ein Regler.

Stolperfalle mit eigenem Block *warnbox*
warnbox ist der eigenständige Block für eine Warnung, die Raum braucht. Ohne Titellargument lautet die Leiste automatisch »Achtung«. **ZSFdanger ist die Inline-Pille.**

Dieselbe Box, anderer Ton

tone=neutral

Jede Box nimmt **tone** als zweites optionales Argument: **chapter** (Default), **neutral** für bewusst kapitelunabhängige Hinweise, **warn** für den Warn-Ton. Ein Regler statt einer neuen Box.

Derselbe Regler, rahmenlos *tone=neutral*
Der Ton wirkt auch dort, wo es keine Kopfzeile gibt: Balken, Titelzeile und Aufzählungszeichen folgen ihm. Im Kapitelton bleibt die Fläche ungetönt, damit die Box dezent bleibt — erst ein gewählter Ton färbt sie.

Regler greifen unabhängig *tone + frame*
tone wählt die Farbwelt, **frame** die Rahmenstärke — beide zusammen, in beliebiger Reihenfolge. Keine Wahl verwirft die andere.

1.2. Polsterung, Rahmen, Umbruch

Die übrigen Regler gelten wortgleich auf jeder Box, und ihre Reihenfolge in der Optionsliste ist bedeutungslos. Das ist keine Absichtserklärung: Jeder Lauf von **make check** setzt jedes Reglerpaar in beiden Reihenfolgen und vergleicht das Ergebnis.

Inhalt bis an die Kante *padx=None, pady=None*
Ohne Innenabstand — so sitzt eine Tabelle bündig im Rahmen und das Zebra endet nicht kurz davor.

Platz für den Balken *padx=bar*
Links mehr Rand: Der Inhalt hält den Akzentbalken der leisen Fassung frei.

Knappste Höhe *pady=tight*
pady regelt die Polsterung oben und unten.

Zwei Regler, eine Box *weight + split*
weight=quiet setzt den Titel als erste Inhaltszeile, **split** teilt den Inhalt. Der Titel steht trotzdem genau einmal — jede Kombination ergibt das, was beide Wahlen zusammen bedeuten.

Kräftige Kontur *frame=hard*
frame wählt nur die Stärke — die Farbe kommt aus dem Ton.

Rahmenlos mit Kopfzeile *frame=None*
Rahmen und Gewicht sind zwei getrennte Fragen.

Bleibt zusammen *atomic*
atomic hält die Box auch im Pack-Modus über der Spaltengrenze zusammen.

Absatzabstand vom Dokument *bodyparskip=inherit*
Statt des eigenen Box-Werts gilt der Dokumentwert. Sichtbar wird das erst am zweiten Absatz.

Eine Stufe kleiner *font=dense*
font=dense setzt den Inhalt eine Stufe kleiner — für gedrängte Register und Aufzählungen, die sonst ein lokales **scriptsize** bräuchten. Der Titel behält seine Grösse: kleiner wird der Inhalt, nicht die Auffindbarkeit. Derselbe Regler heisst an der **ZSFtable** gleich und bedeutet dasselbe.

1.2.1. Dritte Lookup-Ebene

Der **SubsubsectionBar** ist die optionale dritte Ebene, sichtbar kleiner und heller. Diese Box zeigt zugleich: ohne Titellargument entfällt die Kopfzeile ganz — dieselbe Box, ein Argument weniger.

2. Verfahren und Listen

2.1. Nummerierte Schrittfolge

Schritte mit Marke und Text *ordered*
1. **Automatisch gezählt**
Die Nummerierung kommt aus der Umgebung.
2. **Zweiteiliger Schritt**
Fette Marke oben, Erklärung darunter.
3. **Nur bei stabiler Folge**
Sonst ist eine Liste die bessere Wahl.

2.2. Faktenliste ohne und mit Rahmen

- ZSFlist** — Ohne Titellargument rahmenlos und ohne Kopfzeile.
- ZSFitem** — Jeder Eintrag ist ein Paar aus Marke und Erklärung.
- Farbiger Punkt** — Das Aufzählungszeichen trägt die Kapitelfarbe.
- Die Marke ist optional — ein Eintrag ohne sie steht als blosser Aussage da, ohne Trenner, der ins Leere zeigt.

Dieselbe Liste mit Titel *mit Titel*

- Ein Argument** — Der Titel bindet die Liste in eine leise Box.
- Gleiche Einträge** — Marke und Aufzählungszeichen bleiben identisch.

Eine Umgebung, vier Erscheinungsformen: **ZSFlist** entscheidet über Titel und **ordered**, nicht über den Namen.

3. Formeln und Zielketten

3.1. Formelbox und Math-API

textVorBox bindet Kontext an die folgende Box.

Was die Formelbox kann

formulabox

textVorBox — innerhalb der Formelbox bindet es an die Formel:

$$x \in \mathbb{R}, \quad z \in \mathbb{C}, \quad n \in \mathbb{N}, \quad k \in \mathbb{Z}, \quad q \in \mathbb{Q}$$

$$\mathrm{d}x, \quad \|\vec{v}\|, \quad |x|, \quad \operatorname{sgn}(x), \quad \operatorname{grad} f, \quad \operatorname{div} \vec{F}, \quad \operatorname{rot} \vec{F}$$

ZSFsep mit Beschriftung gliedert die Box

$$\sum_{i=1}^n i, \quad \lim_{x \rightarrow 0} \frac{\sin x}{x}$$

textNachBox — hier die Zeile direkt unter der Formel.
formulanote setzt die Anmerkung als eigene Zeile darunter.

textNachBox bleibt optisch an der Box gebunden.

Hervorhebung, Klammer, Zeilenanmerkung

$$a^2 + 2ab + b^2 = (a + b)^2$$

$$\underbrace{x^2 + 2x + 1}_{\text{ZSFbraceunder}} = \overbrace{(x + 1)^2}^{\text{ZSFbraceover}}$$

$$a^2 + b^2 = c^2$$

ZSFformuline: Anmerkung auf der Zeile

3.2. Bedingung, Schritt, Ziel

Zielkette in zwei Schreibweisengoalbox

ZSFDerivationCase* — einzeilig

$\sqrt{x^2} = x$

ZSFDerivationCase

$\sqrt{x^2}$
 $= -x$

Frei aus den Primitiven gebaut

GoalCondition

GoalStep

beliebig oft wiederholbar

GoalTarget

Operatoren des Opt-in-Moduls11_math_advanced

Ker A, rang A, Spur A, diag(A),
span(v₁, v₂), eig(A), proj_U(v),
sig(A), Adj(A), Re z, Im z, *, ⊗.

$A = \underbrace{\begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & 5 \end{pmatrix}}_{\text{Pivotspalten}}$

4. Tabellen und Grids

4.1. Header und Zebra

Farbiger Header, Zebra ab Zeile 2 <table>tbody<tr><td>Element</td><td>Wirkung</td><td>Zeile</td></tr><tr><td>ZSFtable</td><td>Header und Zebra</td><td>1</td></tr><tr><td>ZSFheaderRow</td><td>färbt die Kopfzeile</td><td>1</td></tr><tr><td>ZSFhead</td><td>färbt jede Kopfzeile</td><td>1</td></tr><tr><td>Zebra</td><td>ab der zweiten Zeile</td><td>2</td></tr></table>			Element	Wirkung	Zeile	ZSFtable	Header und Zebra	1	ZSFheaderRow	färbt die Kopfzeile	1	ZSFhead	färbt jede Kopfzeile	1	Zebra	ab der zweiten Zeile	2
Element	Wirkung	Zeile															
ZSFtable	Header und Zebra	1															
ZSFheaderRow	färbt die Kopfzeile	1															
ZSFhead	färbt jede Kopfzeile	1															
Zebra	ab der zweiten Zeile	2															

4.2. Ohne Header oder Zebra

Zebra ab der ersten Zeileheader=false	
header=false	ohne Kopfzeile
Streifen	ab Zeile 1
Einsatz	reine Datenlisten

Ganz ohne Streifenzebra=false	
zebra=false	kein Zebra
Einsatz	sehr kurze Tabellen

Dichte Schrift für lange Tabellenfont=dense		
Regler	Wirkung	Default
header	Kopfzeile	true
zebra	Streifen	true
font	Schriftgröße	normal

4.3. Gitterlinien

Nur waagrechte Liniengrid=horizontal	
grid	both, horizontal, none
Vorbelegung	both
Gleiche Werte	wie am valuegrid

Ganz ohne Liniengrid=none	
grid=none	nur das Zebra trennt
Einsatz	kurze Datenliste

4.4. Zeilenhöhe und Spaltenpolsterung

Mehr Höhe für Zellen mit Brüchenrows=roomy	
Grösse	Term
Steigung	$\frac{\Delta y}{\Delta x}$
Mittel	$\frac{a + b}{2}$

Knappe Zeilen für Kurzinhalterows=tight	
Einzeilig	knapp
Register	dicht
Kurzinhalte	passt

Viele Spalten, knapper Zwischenraumcolsep=tight				
Bauform	Ax	Rad	Win	Ø
Erste	gut	gut	kaum	12
Zweite	kaum	sehr gut	gut	20
colsep=tight wählt die Zellpolsterung — zwischen den Spalten und an der Boxkante zugleich, weil die Farbfläche einer Zeile genau um diesen Wert überragt. Dieser Satz steht in einer ZSFinset und ist deshalb eingerückt, obwohl die Tabelle darüber bis an die Kante läuft.				

4.5. Zellen verbinden

Gruppiert Kopf und AbschnitszeileZSFspan				
Querschnitt	Biegung		Torsion	
Voll	32	16	16	8
Hohlprofil: zusätzlich Innendurchmesser				
Rohr	64	32	32	16

4.6. Bild in der Zelle

Bild ohne eigene BoxZSFimage		
Bauform	Name	Merkmal
	Erste	ohne Höhenangabe
	Zweite	gleich hoch von selbst

Spaltentypen im VergleichL/C/R/F			
L linksbündig	C zentriert	R rechtsbündig	F1 : x ² +1

Kompaktes Wertegittervaluegrid		
valuegrid	Spalte 1	Spalte 2
Argument	Anzahl	Datenspalten
Erste Spalte	immer	Beschriftung

Dasselbe Gitter, weniger Liniengrid		
grid	horizontal	nur Zeilen
Vorbelegung	both	alle Linien
Dritter Wert	none	gar keine

Ganz ohne Gittergrid=none		
Zuordnung	links	rechts
Wo die Spalten	selbst	sprechen

4.7. Mehrere Boxen als ein Block

Blöcke mit gleicher Spaltenstruktur brauchen die Kopfzeile nur einmal. Die **Boxgruppe** setzt sie über die ganze Gruppe und schliesst die Zwischenräume — der Stapel liest sich als eine Tabelle, jeder Block behält aber seinen eigenen Titel.

Symbol	Bedeutung	Einheit
1. Erster Block		
E	Feldstärke	V/m
U	Spannung	V
2. Zweiter Block		
I	Stromstärke	A

Warum die Aufteilung an der Gruppe steht
cols wird einmal gesetzt, die Mitglieder lesen sie als ZSFgroupcols. In jedem Block wiederholt stünde dieselbe Entscheidung mehrfach — und beim n-ten einmal anders.
Alle Spaltenbreiten sind proportional: die Faktoren einer Tabelle müssen zur Anzahl X-Spalten summieren, dann füllt sie die Spalte exakt aus.

5. Abbildungen und Layout

5.1. Bild-Makros

Ein Bild, optional skaliertZSFfig



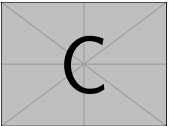
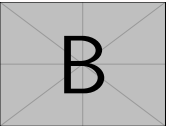
width und height sind Schlüssel wie überall; ohne sie greifen die zentralen Vorbelegungen.

Bild links, Text rechtsZSFfigside



Der erste Wert bestimmt den Bildanteil, der Rest gehört dem Text.

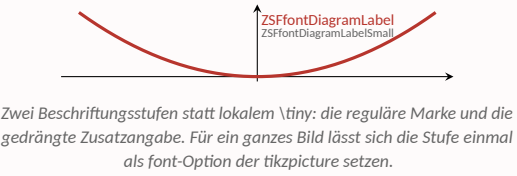
Dasselbe Makro, mehrere PfadeZSFfig



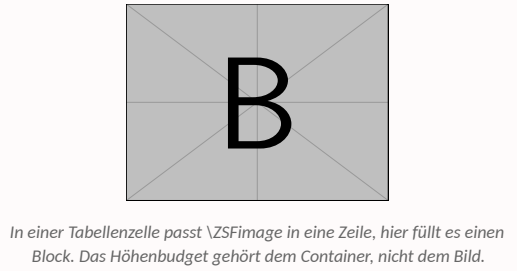
Eine Pfadliste ergibt eine Reihe; die Breite folgt der Anzahl.

5.2. Plot ohne externe Datei

Diagramm direkt im Dokument figbox + pgfplots



Dasselbe Bild-Makro, anderer Container ZSFImage in figbox



Derselbe Regler, mit Kopfzeile split

Formel	split ist ein Regler wie jeder andere und gilt deshalb auch auf einer Box mit Titel — vorher brauchte es dafür eine Verschachtelung.
linker Block	splitbox ordnet zwei beliebige Inhaltsblöcke ohne sichtbaren Rahmen nebeneinander an; split legt den linken Anteil fest, splitalign die Ausrichtung der Hälften zueinander. Dieselbe Box auf Grundlinie statt Mitte. Formel neben Erklärung will meist die Mitte, Diagramm neben Legende die Oberkante.
linker Block	

6. Navigation und Semantik

6.1. Scan-Anker im Inhalt

Ein **Schlagwort** ist der primäre Scan-Anker. Die **prüfungskritischste Aussage** darf zusätzlich hervorgehoben werden. **Vorzeichen prüfen** markiert eine lokale Gefahr. ⇒ Das Resultat ist sofort auffindbar.

Fett ohne Fachbegriff-Semantik
ZSFlabel: eine neutrale Beschriftung — Listenkopf, Zeilenmarke, Linksammlung. Kein Registereintrag. Sie sieht aus wie **ZSFkeyword** und ist es doch nicht: Der Unterschied liegt in der Bedeutung, nicht im Satz. Inline-Betonung trägt bewusst keine Farbe — die bleibt den Flächen, den Verweisen und den Formel-Markern.

Navigierbarer Titel ZSFTitleTag

Fernverweis zur Tabellenübersicht: (→ 4.1). Kompakte Zielnummer: 3.2. Ein Skriptverweis bleibt rechtsbündig. (S.42)

Unnummerierte Struktur

Sternvariante
SubsectionBar* schafft Orientierung ohne neuen Locator.
Unnummerierte dritte Ebene
Nummerierte Kapitel, Abschnitte und Unterabschnitte liefern Labels für papierfeste Verweise; Sternvarianten bleiben rein visuell.

6.2. Lesbarkeit in schmalen Spalten

Flattersatz, Umbruch-Penalties und Microtype wirken automatisch. Mit **ZSFbbreak**, **ZSFnobreak** und **ZSFallowbreak** bleiben gezielte Ausnahmen möglich.

7. Code und Kommentare

7.1. CodeExpert-Listing

Was der Listing-Stil einfärbt codebox

```
# Kommentar: gruen gesetzt
def hervorhebung(text="Zeichenkette",
    zahl=42):
    if zahl > 0:
        return text
    return None
```

7.2. Teilbare Code-Gruppe

Teil 1 trägt den Titel part=first

```
# Rahmen oben, unten offen
werte = [1, 2, 3]

# part=mid: beide Kanten offen
summe = sum(werte)

# part=last: Rahmen unten
print(summe)
```

7.3. Smarte Zeilenkommentare

Kommentar rechts oder darüber CodeLine
kurz = 1 **#passt rechts daneben**
#zu breit, deshalb darüber
sehr_langer_variablenname = 1
limit = 10 **#InlineComment: erzwungen rechts**
#OverlineComment: erzwungen darüber
fertig = True

```
# codebox ohne Titellargument: keine
    Kopfzeile
titellos = True
```

- **CodeLine** — Entscheidet nach Breite, wo der Kommentar steht.
- **SetCodeCommentThreshold** — Verschiebt die Schwelle für den Umbruch.
- **ResetCodeCommentThreshold** — Stellt den Standard wieder her.

8. Stilmittel

Diese Mittel kosten keinen Platz, sparen aber Lesezeit. Sie lohnen sich überall dort, wo eine Umformung sonst Zeile für Zeile nachvollzogen werden müsste.

8.1. Terme über Umformungen verfolgen

Wohin wandert welcher Koeffizient?

$$ax^2 + bx + c = 0$$
$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Drei Gegebene, fünf Fundstellen im Resultat — mit Farbe ist die Zuordnung sofort da, ohne Farbe muss man sie jedes Mal nachlesen.

- **ZSFmhIA/B/D** — Die gegebenen Größen, je ein Strang.
- **ZSFmhIC** — Das Ziel — immer die Endform oder das Resultat.
- **Gleiche Farbe** — bedeutet immer dieselbe Grösse, nie bloss Betonung.

8.2. Grössenfarben: Farbe als Identität

Die Marker oben sind **positional** — sie gelten innerhalb einer Herleitung. Wo dieselben Größen über das ganze Dokument hinweg auftauchen, gibt es das Gegenstück: eine Farbe gehört dauerhaft einer **Grösse**.

Dieselbe Farbe, quer durch die ZSF

$$M = F \cdot r$$

Andere Stelle, andere Formel

$$W = F \cdot s \quad F = \frac{M}{r}$$

Kraft, Weg und Moment behalten ihre Farbe in jeder Formel. Vergeben werden sie einmal in `preamble.tex`, nie im Kapitel.

Zwei Systeme, zwei Fragen
ZSFmhIA..D beantwortet »welche Rolle spielt dieser Term in dieser Umformung«. **ZSFDeclareQuantity** beantwortet »welche Grösse ist das, überall«. Beide gleichzeitig zu verwenden ist möglich, aber selten sinnvoll — zwei Farbsprachen in einer Formel liest niemand unter Zeitdruck.

8.3. Formelteile benennen

Der Name steht am Teil, nicht daneben

$$\underbrace{x^2 + 2x + 1}_{\text{Ausgangsform}} = \overbrace{(x + 1)^2}^{\text{faktorisiert}}$$
$$\sum_{k=1}^n k = \frac{n(n+1)}{2} \quad \text{geschlossene Form}$$

Eine Note unter der Box müsste erst sagen, welcher Teil gemeint ist. Die Klammer zeigt direkt darauf.

8.4. Zusammenspiel

Farbe, Klammer und Hervorhebung zusammen

$$\underbrace{uv}' = u'v + uv'$$

Produkt

Farbe trägt die Zuordnung, die Klammer den Namen, die Hervorhebung die Kernaussage.

Wann es sich nicht lohnt

Farbe nur setzen, wenn die Zuordnung mathematisch eindeutig ist. Zur blossen Betonung eines Terms ist sie falsch — dafür gibt es **ZSFdanger** und ZSFhl. Im Zweifel keine Farbe.

9. Palette Slot 9

9.1. Abschnittston

9.1.1. Unterabschnittston

Box-Akzent
Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

10. Palette Slot 10

10.1. Abschnittston

10.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

11. Palette Slot 11

11.1. Abschnittston

11.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

12. Palette Slot 12

12.1. Abschnittston

12.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

13. Palette Slot 13

13.1. Abschnittston

13.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

14. Palette Slot 14

14.1. Abschnittston

14.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

15. Palette Slot 15

15.1. Abschnittston

15.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

16. Palette Slot 16

16.1. Abschnittston

16.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

17. Palette Slot 17

17.1. Abschnittston

17.1.1. Unterabschnittston

Box-Akzent

Kapitel-, Abschnitts- und Hintergrundfarbe im direkten Vergleich.

Stichwortverzeichnis

A

Achtung *siehe* warnbox 1.1

B

Bild in Tabelle 4.6 · (S. 2)

Boxgruppe 4.7 · (S. 2)

C

CodeLine 7.3 · (S. 3)

D

defbox 1.1 · (S. 1)

F

Fernverweis 6.1 · (S. 3)

figbox 5.2 · (S. 3)

formulabox 3.1 · (S. 1)

G

Gitterlinien 4.3 · (S. 2)

goalbox 3.2 · (S. 2)

Grösse 8.2 · (S. 3)

Grössenfarbe 8.2 · (S. 3)

K

Klammer-Annotation 8.3 · (S. 3)

L

Lesbarkeit 6.2 · (S. 3)

M

Math-API 3.1 · (S. 1)

R

runintext 1.1 · (S. 1)

S

Scan-Anker 6.1 · (S. 3)

Schlagwort 6.1 · (S. 3)

Schnellzugriff [Start](#)

Spaltenpolsterung 4.4 · (S. 2)

splitbox 5.2 · (S. 3)

SubsubsectionBar 1.2.1 · (S. 1)

T

Term-Einfärbung 8.1 · (S. 3)

V

valuegrid 4.6 · (S. 2)

W

warnbox 1.1 · (S. 1)

Z

Zeilenhöhe 4.4 · (S. 2)

Zellverbund 4.5 · (S. 2)

ZSFfig 5.1 · (S. 2)

ZSFfigside 5.1 · (S. 2)

ZSFinset 4.4 · (S. 2)

ZSFlist 2.2 · (S. 1)

ZSFtable 4.1 · (S. 2)

Schlusswort

Das **Hliddal Template** bildet die gemeinsame Grundlage meiner Zusammenfassungen. Es ist für kompakte, schnell erfassbare Prüfungsunterlagen entwickelt und kann als Ausgangspunkt für eigene ZSF verwendet und angepasst werden. Viel Erfolg beim Erstellen deiner eigenen Zusammenfassung! Fehler, Verbesserungsvorschläge oder Ergänzungen gerne als Issue melden.

Links & Ressourcen:

Aktuelle Version & Hliddal Template:

garcon-studios.ch/zsf

Hliddal Template auf GitHub: github.com/1hliddal/zsf-template-hliddal